Informe de Laboratorio: Estructura de Computadores

Nombre del Estudiante: [Jaime Andres Aricapa Cruz]
Fecha: [F2026/03/02]
Asignatura: Estructura de Computadores

Enlace del repositorio en GitHub: [agregar enlace Aquí]

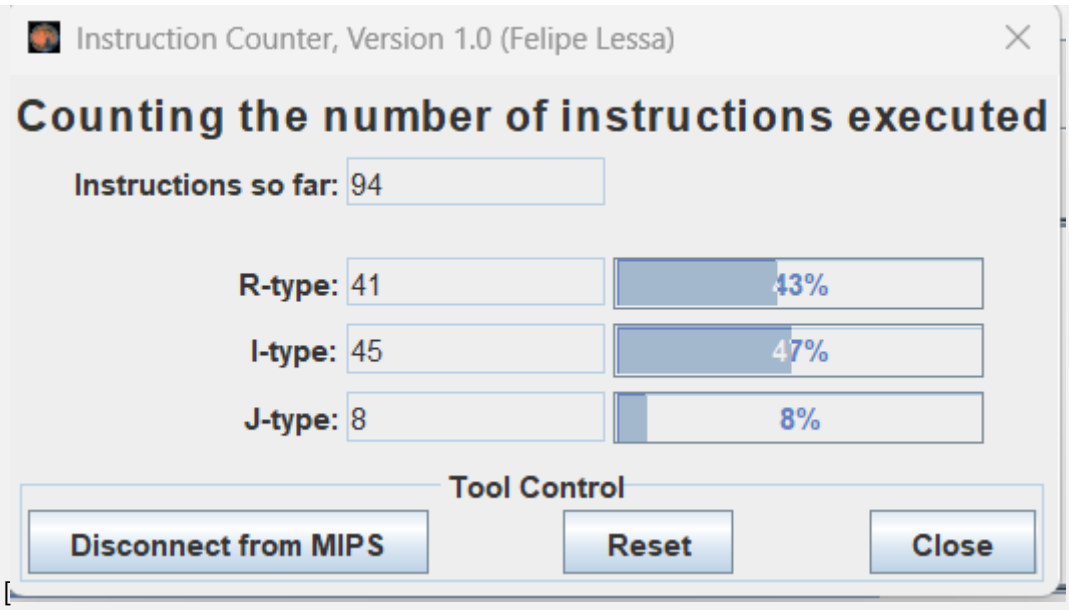
---

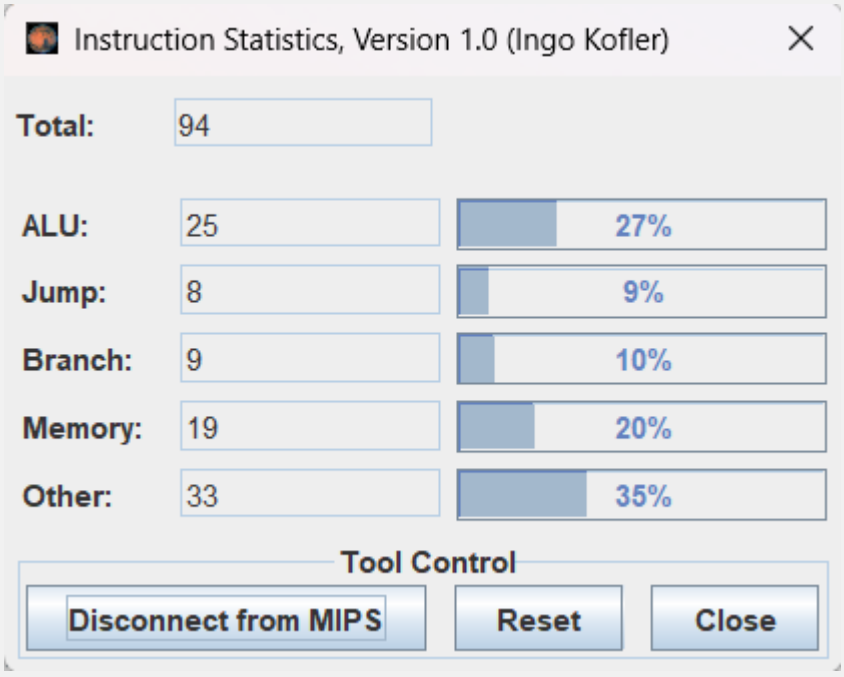
1. Análisis del Código Base

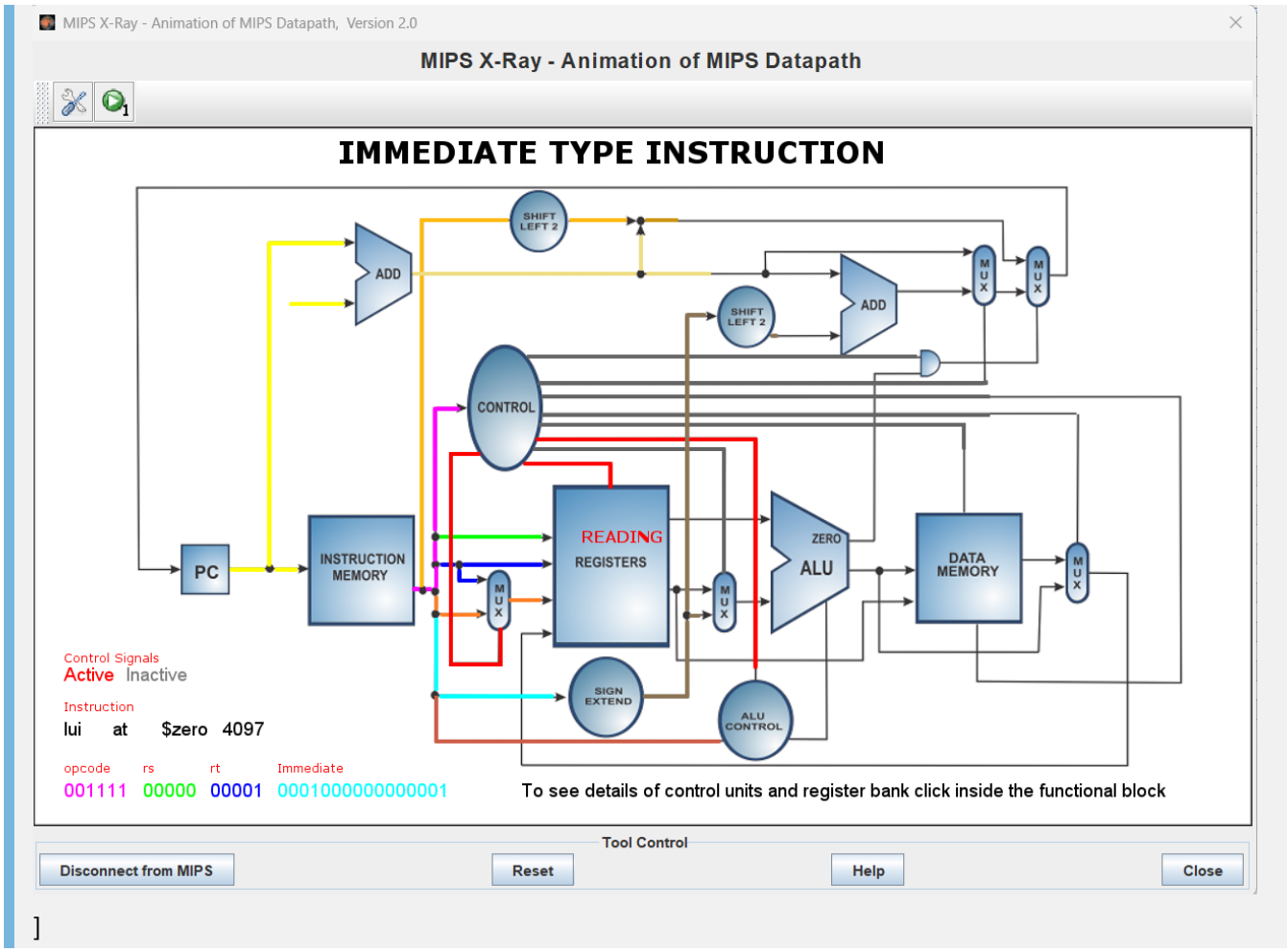
1.1. Evidencia de Ejecución

Adjunte aquí las capturas de pantalla de la ejecución del `programa_base.asm` utilizando las siguientes herramientas de MARS:

- **MIPS X-Ray** (Ventana con el Datapath animado).
- **Instruction Counter** (Contador de instrucciones totales).
- **Instruction Statistics** (Desglose por tipo de instrucción).


Instruction Counter, Version 1.0 (Felipe Lessa)
Counting the number of instructions executed
Instructions so far: 94
R-type: 41
43%
I-type: 45
47%
J-type: 8
8%
Tool Control
Disconnect from MIPS
Reset
Close


Instruction Statistics, Version 1.0 (Ingo Kofler)
Total: 94
ALU: 25
27%
Jump: 8
9%
Branch: 9
10%
Memory: 19
20%
Other: 33
35%
Tool Control
Disconnect from MIPS
Reset
Close



1.2. Identificación de Riesgos (Hazards)

Completa la siguiente tabla identificando las instrucciones que causan paradas en el pipeline:

Instrucción Causante	Instrucción Afectada	Tipo de Riesgo (Load-Use, etc.)	Ciclos de Parada
`lw $t6, 0($t5)`	`mul $t7, $t6, $t0`	Load-Use	

1.2. Estadísticas y Análisis Teórico

Dado que MARS es un simulador funcional, el número de instrucciones ejecutadas será igual en ambas versiones. Sin embargo, en un procesador real, el tiempo de ejecución (ciclos) varía. Completa la siguiente tabla de análisis teórico:

Métrica	Código Base	Código Optimizado
Instrucciones Totales (según MARS)	94	94
Stalls (Paradas) por iteración	1	0
Total de Stalls (8 iteraciones)	8 (1 por cada lw-mul)	0
Ciclos Totales Estimados (Inst + Stalls)	102	94
CPI Estimado (Ciclos / Inst)	1.085	1

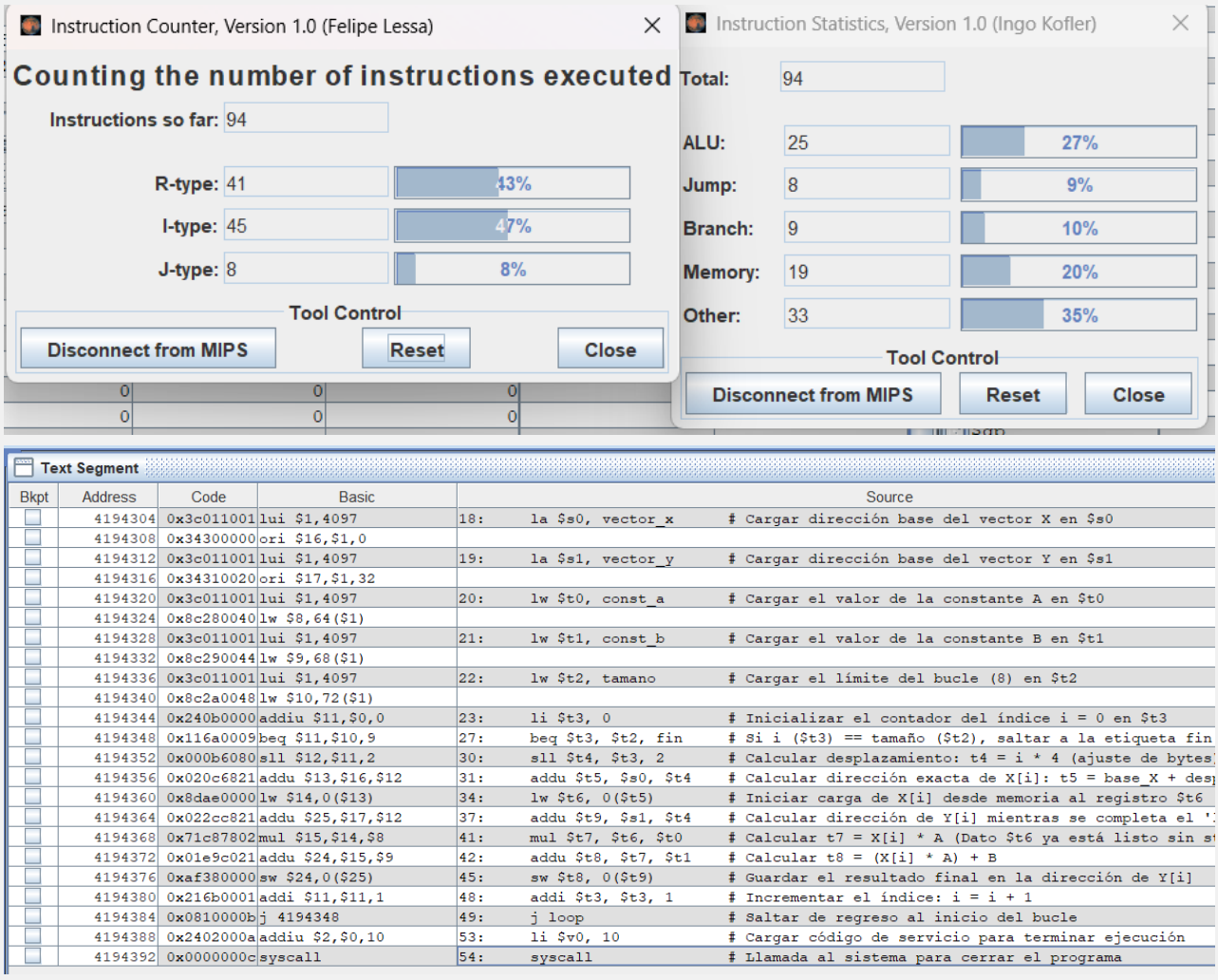
2. Optimización Propuesta

2.1. Evidencia de Ejecución (Código Optimizado)

Adjunte aquí las capturas de pantalla de la ejecución del `programa_optimizado.asm` utilizando las mismas herramientas que en el punto 1.1:

- **MIPS X-Ray**.
- **Instruction Counter**.
- **Instruction Statistics**.

[



Bkpt	Address	Code	Basic		Source
	4194304	0x3c011001	lui $1,4097	18: la $s0, vector_x	# Cargar dirección base del vector X en $s0
	4194308	0x34300000	ori $16,$1,0		
	4194312	0x3c011001	lui $1,4097	19: la $s1, vector_y	# Cargar dirección base del vector Y en $s1
	4194316	0x34310020	ori $17,$1,32		
	4194320	0x3c011001	lui $1,4097	20: lw $t0, const_a	# Cargar el valor de la constante A en $t0
	4194324	0x8c280040	lw $8,64($1)		
	4194328	0x3c011001	lui $1,4097	21: lw $t1, const_b	# Cargar el valor de la constante B en $t1
	4194332	0x8c290044	lw $9,68($1)		
	4194336	0x3c011001	lui $1,4097	22: lw $t2, tamano	# Cargar el límite del bucle (8) en $t2
	4194340	0x8c2a0048	lw $10,72($1)		
	4194344	0x240b0000	addiu $11,$0,0	23: li $t3, 0	# Inicializar el contador del índice i = 0 en $t3
	4194348	0x116a0009	beq $11,$10,9	27: beq $t3, $t2, fin	# Si i ($t3) == tamaño ($t2), saltar a la etiqueta fin
	4194352	0x000b6080	sll $12,$11,2	30: sll $t4, $t3, 2	# Calcular desplazamiento: t4 = i * 4 (ajuste de bytes)
	4194356	0x020c6821	addu $13,$16,$12	31: addu $t5, $s0, $t4	# Calcular dirección exacta de X[i]: t5 = base_X + desp
	4194360	0x8dae0000	lw $14,0($13)	34: lw $t6, 0($t5)	# Iniciar carga de X[i] desde memoria al registro $t6
	4194364	0x022cc821	addu $25,$17,$12	37: addu $t9, $s1, $t4	# Calcular dirección de Y[i] mientras se completa el '
	4194368	0x71c87802	mul $15,$14,$8	41: mul $t7, $t6, $t0	# Calcular t7 = X[i] * A (Dato $t6 ya está listo sin s
	4194372	0x01e9c021	addu $24,$15,$9	42: addu $t8, $t7, $t1	# Calcular t8 = (X[i] * A) + B
	4194376	0xaf380000	sw $24,0($25)	45: sw $t8, 0($t9)	# Guardar el resultado final en la dirección de Y[i]
	4194380	0x216b0001	addi $11,$11,1	48: addi $t3, $t3, 1	# Incrementar el índice: i = i + 1
	4194384	0x0810000b	j 4194348	49: j loop	# Saltar de regreso al inicio del bucle
	4194388	0x2402000a	addiu $2,$0,10	53: li $v0, 10	# Cargar código de servicio para terminar ejecución
	4194392	0x0000000c	syscall	54: syscall	# Llamada al sistema para cerrar el programa

]

2.2. Código Optimizado

Pega aquí el fragmento de tu bucle `loop` reordenado:

```
# loop:
    # --- Condición de salida ---
    beq $t3, $t2, fin      # Si i ($t3) == tamaño ($t2), saltar a la etiqueta fin

    # --- Preparación de direcciones ---
    sll $t4, $t3, 2        # Calcular desplazamiento: t4 = i * 4 (ajuste de bytes)
    addu $t5, $s0, $t4     # Calcular dirección exacta de X[i]: t5 = base_X + despl
```

```
    # --- Carga y Optimización (Reordenamiento) ---
    lw $t6, 0($t5)        # Iniciar carga de X[i] desde memoria al registro $t6

    # ESTA INSTRUCCIÓN HA SIDO MOVIDA AQUÍ:
    addu $t9, $s1, $t4    # Calcular dirección de Y[i] mientras se completa el
'lw'
    # Al colocar esta instrucción aquí, llenamos el hueco del stall de Load-Use.

    # --- Operaciones aritméticas ---
    mul $t7, $t6, $t0     # Calcular t7 = X[i] * A (Dato $t6 ya está listo sin
stall)
    addu $t8, $t7, $t1    # Calcular t8 = (X[i] * A) + B

    # --- Almacenamiento ---
    sw $t8, 0($t9)        # Guardar el resultado final en la dirección de Y[i]

    # --- Control de bucle ---
    addi $t3, $t3, 1      # Incrementar el índice: i = i + 1
    j loop                # Saltar de regreso al inicio del bucle
```

2.2. Justificación Técnica de la Mejora

Explica qué instrucción moviste y por qué colocarla entre el `lw` y el `mul` elimina el riesgo de datos:

> La instrucción seleccionada fue: addu $t9, $s1, $t4 (Cálculo de la dirección de destino para $Y[i]$)

¿Por qué esta instrucción? Para que el reordenamiento funcione, la instrucción elegida debe cumplir dos requisitos:

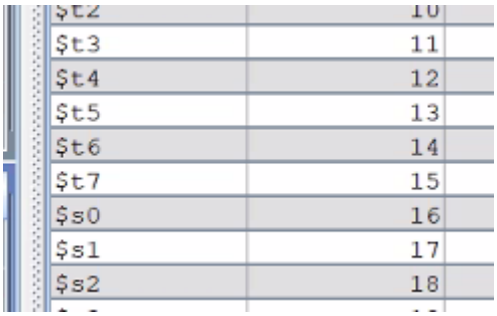
Ser independiente: No debe necesitar el resultado del lw (que está en $t6) ni el resultado del mul (que estará en $t7).

Ser necesaria: Debe estar dentro del bucle para que realmente "rellene" el hueco en cada iteración.

---

3. Comparativa de Resultados

Métrica	Código Base	Código Optimizado	Mejora (%)
Ciclos Totales	94	94	0
Stalls (Paradas)	8	0	100%
CPI	1.085	1	7,84%



$t2	10
$t3	11
$t4	12
$t5	13
$t6	14
$t7	15
$s0	16
$s1	17
$s2	18

4. Conclusiones

¿Qué impacto tiene la segmentación en el diseño de software de bajo nivel? ¿Es siempre posible eliminar todas las paradas?

> [La segmentación (pipeline) cambia drásticamente la forma en que debemos escribir código en lenguaje ensamblador. En un procesador no segmentado, el orden de las instrucciones no afecta el tiempo de ejecución (siempre que la lógica sea correcta). Sin embargo, en arquitecturas segmentadas como MIPS, el programador de bajo nivel debe ser consciente de la latencia de las instrucciones.

El impacto principal es que el rendimiento ya no depende solo de qué instrucciones se usan, sino de en qué orden se colocan. Como vimos en el laboratorio, un código lógicamente idéntico puede ser un 7.84% más lento simplemente por tener una dependencia de datos mal ubicada. Esto obliga al diseñador de software (o al compilador) a aplicar técnicas de Planificación de Instrucciones para mantener el "flujo" del pipeline constante y evitar que el hardware desperdicie ciclos de reloj.

¿Es siempre posible eliminar todas las paradas (stalls)? No, no siempre es posible eliminar todas las paradas por completo. Existen limitaciones fundamentales:Falta de instrucciones independientes: En algoritmos muy lineales donde cada paso depende estrictamente del anterior (ejemplo: $A \rightarrow B \rightarrow C \rightarrow D$), no existen instrucciones "libres" que podamos mover para rellenar los huecos del pipeline.

Riesgos de Control: Las instrucciones de salto (beq, j) pueden causar paradas si el procesador no sabe a qué dirección saltar hasta que la instrucción se decodifica o ejecuta. Aunque existen técnicas como la "predicción de saltos", no siempre son efectivas al 100%.

Riesgos Estructurales: A veces, dos instrucciones necesitan usar el mismo recurso físico (como la memoria o un puerto de escritura) al mismo tiempo, lo que obliga a una de ellas a esperar.Acceso a Memoria Cache (Misses): Si un dato no está en la memoria caché, el stall generado por la espera a la memoria RAM principal es de decenas o cientos de ciclos, algo que es imposible de ocultar simplemente reordenando un par de instrucciones]